

ILearn 代码仓库与工程材料说明

项目：ILearn (课标在环 · 多 Agent 协同 · 自适应学习引擎)
赛事：2026 世界人工智能开源大赛 (GOAI 2026) | 赛道二 AI+教育
仓库：https://github.com/QinHsiu/ILearn
更新日期：2026-09-06

1. 运行入口

项目提供两种运行入口：

入口一：API 服务 (Web 模式)

```
# 启动后端 API 服务
uvicorn ilearn.api.app:app --reload --host 127.0.0.1 --port 8000

# 启动前端开发服务 (另开终端)
cd frontend && npm run dev
```

入口二：CLI 命令行 (离线模式)

```
# 零 LLM 离线运行完整闭环
python -m ilearn.cli.main agents run --region 北京 --grade 5 --age 11 --offline
```

核心入口文件

入口	路径	说明
后端入口	ilearn/api/app.py	FastAPI 应用
CLI 入口	ilearn/cli/main.py	命令行工具
前端入口	frontend/src/main.tsx	React 应用

2. 依赖说明

后端依赖 (requirements.txt / pyproject.toml)

依赖包	版本	用途
fastapi	>=0.104.0	Web 框架
uvicorn	>=0.24.0	ASGI 服务器
pydantic	>=2.0.0	数据验证
qdrant-client	>=1.7.0	向量检索（可选）
weasyprint	>=60.0	PDF 高清渲染
fpdf2	>=2.7.0	PDF 备选引擎
pytest	>=7.0.0	测试框架
httpx	>=0.25.0	API 测试客户端

前端依赖 (frontend/package.json)

依赖包	版本	用途
react	>=18.0.0	UI 框架
vite	>=4.0.0	构建工具
typescript	>=5.0.0	类型检查
vitest	>=0.34.0	单元测试

安装命令

```
# 后端
pip install -e ".[dev]"

# 前端
cd frontend && npm install
```

3. 配置文件

配置文件	路径	用途
项目配置	pyproject.toml	项目元数据、依赖、工具配置
模型路由	ilearn/config/model_routing.json	LLM 模型路由配置

配置文件	路径	用途
能力注册	ilearn/config/capabilities.json	系统能力声明（离线/在线）
前端配置	frontend/vite.config.ts	Vite 构建配置
TypeScript 配置	frontend/tsconfig.json	TS 编译配置

关键配置示例 (model_routing.json)

```
{
  "default": {
    "provider": "openai",
    "model": "gpt-4o-mini",
    "temperature": 0.3
  },
  "diagnosis": {
    "provider": "openai",
    "model": "gpt-4o",
    "temperature": 0.1
  },
  "offline": {
    "enabled": true,
    "fallback": "rule_based"
  }
}
```

4. 示例数据

数据类型	路径	说明
演示单元	ilearn/data/demo_units/math_5_1.py	五年级小数乘法完整演示数据
知识图谱	ilearn/data/knowledge_graph/	小学数学知识图谱
课标数据	ilearn/data/curriculum/	人教版课标映射数据
测试数据	tests/fixtures/	单元测试 fixtures

演示数据说明 (math_5_1)

- 知识点： 小数乘整数、小数乘小数、积的近似数、运算律推广
- 预设画像： 模拟中等水平学生（掌握度 45%–85%）
- 模拟班级： 35 名学生，平均掌握度 62%
- 一键体验： 无需注册，点击即可进入完整闭环

5. 部署说明

本地部署（开发/演示）

```
# 1. 克隆仓库
git clone https://github.com/QinHsiu/ILearn.git
cd ILearn

# 2. 安装依赖
pip install -e ".[dev]"
cd frontend && npm install && cd ..

# 3. 启动服务
# 终端1: uvicorn ilearn.api.app:app --reload --host 127.0.0.1 --port 8000
# 终端2: cd frontend && npm run dev

# 4. 访问 http://127.0.0.1:5173
```

生产部署（可选）

```
# 使用 Docker（如有 Dockerfile）
docker build -t ilearn .
docker run -p 8000:8000 -p 5173:5173 ilearn

# 或使用 gunicorn
gunicorn -w 4 -k uvicorn.workers.UvicornWorker ilearn.api.app:app
```

环境变量

变量名	说明	默认值
OPENAI_API_KEY	OpenAI API 密钥（可选）	—
ILEARN_PDF_BACKEND	PDF 引擎选择	weasyprint
ILEARN_OFFLINE_MODE	离线模式开关	false

6. 测试方法

后端测试（554+ tests）

```
# 运行所有测试
pytest -q

# 运行特定模块
```

```
pytest tests/test_core/ -v

# 运行带覆盖率
pytest --cov=ilearn --cov-report=html
```

前端测试 (73+ tests)

```
cd frontend
npm run test
npm run test:coverage
```

集成测试

```
# API 链路测试
python tests/test_integration.py

# 端到端测试 (需服务运行)
pytest tests/test_e2e/
```

7. 运行证据

证据类型	文件/命令	说明
测试通过日志	pytest -q --tb=no	554+ tests passed
CLI 离线运行	python -m ilearn.cli.main agents run --offline	完整闭环输出
API 健康检查	curl http://127.0.0.1:8000/health	返回 {"status":"ok"}
演示会话创建	curl -X POST /demo/units/math_5_1 /session	返回 session_id
前端构建	cd frontend && npm run build	构建成功无报错

证据截图与运行日志存放位置: runtime_evidence/ 目录。